

Retrieval-Augmented Generation (RAG)

Section A — Concept Application

SCENARIO 1 : You are building a customer support chatbot for a food delivery app. During testing, you notice that the chatbot gives inconsistent answers — sometimes it refuses valid refund requests, and sometimes it approves them without verifying order status. Your team traces the problem to vague prompts that do not specify the expected output format or the decision rules the LLM should follow .

Question: Explain what distinguishes a clear prompt from an unclear one in this context. Describe at least two specific changes you would make to the existing prompts to produce consistent, reliable responses from the LLM, and justify why each change matters .

Ans : A clear prompt provides specific instructions about the chatbot's role, the rules it must follow, the information it should check, and the expected response format. An unclear prompt is vague and does not define these details, causing the LLM to make assumptions and produce inconsistent responses.

Two changes to improve the prompts

1. Add clear decision rules

- Specify exactly when a refund should be approved, denied, or when more information is needed.
- Example:
 - Approve refunds only if the order is cancelled or not delivered.
 - Deny refunds for delivered orders unless there is a verified issue.
 - Ask for the order ID if order details are missing.

Why it matters:

Clear rules reduce ambiguity and ensure the chatbot makes consistent decisions based on company policy instead of guessing.

2. Define a standard output format

- Require every response to follow a fixed structure, such as:
 - Decision: Approved / Denied / More Information Needed
 - Reason: Explain why.
 - Next Step: Tell the customer what to do next.

Why it matters:

A standard format makes responses consistent, easy for customers to understand, and simpler to integrate with the support system.

Conclusion :

A clear prompt contains explicit instructions, decision rules, and a structured response format, while an unclear prompt leaves these unspecified. Adding clear refund criteria and a fixed output format will produce more consistent, reliable, and accurate responses from the LLM.

SCENARIO 2 : You are designing a complaint classifier for a food delivery platform. The classifier must label incoming customer messages into one of four categories: 'Late Delivery', 'Wrong Item', 'Missing Item', and 'Poor Quality'. You have 200 labelled examples available, but the LLM you are using has a limited context window and cannot accept all of them at once.

Question: Compare zero-shot and few-shot prompting as strategies for building this classifier. Justify which approach you would choose given the constraints described, and explain what factors determine how many examples to include in a few-shot prompt .

Ans : Zero-shot prompting means giving the LLM only instructions to classify customer complaints without providing any examples. The model relies on its existing knowledge to assign one of the four categories. This approach is simple and uses very little context, but it may produce less accurate or inconsistent classifications, especially when customer messages are ambiguous.

Few-shot prompting provides the LLM with a small number of labelled examples before the new complaint. These examples demonstrate how messages should be classified, helping the model understand the expected categories and improving accuracy and consistency.

Which approach should be chosen?

Given the constraints, few-shot prompting is the better choice. Although there are 200 labelled examples, the LLM's limited context window prevents including them all. Instead, a carefully selected set of representative examples from each category can be used. This gives the model enough guidance while staying within the context limit, resulting in more reliable classifications than zero-shot prompting.

Factors that determine how many examples to include

1. Context window size – The total prompt, including instructions, examples, and the customer's message, must fit within the LLM's context limit.
2. Task complexity – More complex or similar categories may require more examples for the model to distinguish between them.
3. Category coverage – Include examples from all four categories so the model learns each class equally.
4. Example quality – A few clear, diverse, and representative examples are usually more effective than many repetitive ones.

Conclusion:

While zero-shot prompting is more efficient in terms of context usage, few-shot prompting is the preferred approach because it provides examples that improve the classifier's accuracy and consistency while still fitting within the LLM's context window.

SCENARIO 3 : You are developing an AI assistant that helps delivery operations staff calculate the optimal stop sequence for a delivery agent given five order addresses, estimated traffic delays, and priority levels. When you send a direct question to the LLM asking for the best route, the answers are frequently incorrect and contain no explanation of the reasoning.

Question: Explain how Chain-of-Thought (CoT) prompting would improve the quality of the assistant's output for this task. Write a brief example showing how you would restructure the prompt to invoke step-by-step reasoning, and identify one limitation of this technique.

Ans: Chain-of-Thought (CoT) prompting improves the quality of the assistant's output by encouraging the LLM to solve the problem step by step instead of giving an immediate answer. For a delivery route, the model can first evaluate the priority of each order, consider traffic delays, compare possible stop sequences, and then choose the optimal route. This structured reasoning reduces errors and makes the final recommendation more logical and easier to understand.

Example of a CoT Prompt :

"You are a delivery route planning assistant. Given five delivery addresses, their priority levels, and estimated traffic delays, solve the problem step by step. First, identify the highest-priority deliveries. Next, evaluate traffic conditions and compare possible routes. Finally, recommend the optimal stop sequence and briefly explain why it is the best choice."

Limitation of Chain-of-Thought Prompting :

Chain-of-Thought prompting produces longer responses and uses more tokens, making it slower and more expensive. It also does not guarantee the optimal or correct solution for complex routing problems.

Conclusion: CoT prompting helps the LLM produce more logical and reliable outputs through step-by-step reasoning, but it increases response length and computational cost.

SCENARIO 4 : You are working on an internal tool that lets delivery partners query a 150-page policy handbook — covering payout rules, SLA terms, and dispute resolution — that is updated every quarter. A colleague proposes fine-tuning an LLM on the handbook, while you propose building a RAG system instead .

Question: Justify your choice of RAG over fine-tuning for this specific use case. Address at least two reasons, including how each approach handles frequent document updates, and identify one scenario in which fine-tuning would have been the better choice .

Ans: For this use case, Retrieval-Augmented Generation (RAG) is a better choice than fine-tuning because the delivery policy handbook is 150 pages long and is updated every quarter. RAG retrieves the latest relevant information from the handbook before generating a response, ensuring answers are accurate and up to date.

Reasons for Choosing RAG

1. Handles frequent document updates easily

- With RAG, when the handbook is updated, only the document database or knowledge base needs to be refreshed.
- With fine-tuning, the LLM must be retrained or fine-tuned again whenever the handbook changes, which is time-consuming and expensive.

Why it matters:

RAG always provides responses based on the latest version of the handbook without retraining the model.

2. Provides accurate and source-based answers

- RAG retrieves only the relevant sections of the handbook and uses them to answer the user's query.
- Fine-tuning stores knowledge in the model's parameters, which can become outdated and may generate incorrect or hallucinated information.

Why it matters:

RAG improves reliability and ensures responses are based on the current policy document.

When Fine-Tuning Would Be Better :

Fine-tuning would be a better choice if the goal were to teach the model a specialized writing style, response format, or domain-specific behavior that rarely changes. For example, training a customer support chatbot to always respond in a company's specific tone or follow a consistent conversation style.

Conclusion

For a frequently updated policy handbook, RAG is the better choice because it retrieves the latest information without retraining and provides accurate, evidence-based answers. Fine-tuning is more suitable when the objective is to learn stable behaviors or communication styles rather than frequently changing knowledge.

SCENARIO 5: You are building a RAG pipeline to let customers search across 500 restaurant menus stored as PDF files. Each menu averages 8 pages. During evaluation you find that retrieved chunks often cut mid-sentence or omit critical context — for example, a price appears in a sentence one paragraph after the dish name, but both rarely appear in the same chunk .

Question: Explain the role of chunk size and chunk overlap in a RAG pipeline. Describe how you would adjust these two parameters to fix the retrieval quality problems described, and identify one trade-off your adjustment introduces .

Ans: In a RAG (Retrieval-Augmented Generation) pipeline, chunk size is the amount of text split into each chunk, while chunk overlap is the number of words or tokens shared between consecutive chunks. These parameters affect how much context is available during retrieval.

Role of Chunk Size and Chunk Overlap

- **Chunk Size:** Determines how much information is stored in each chunk. Larger chunks contain more context, while smaller chunks are more focused but may split related information.
- **Chunk Overlap:** Ensures that important information appearing near the boundary of one chunk is also included in the next chunk, reducing the chance of losing context.

How to Improve Retrieval Quality

1. Increase the chunk size

- Use larger chunks so that related information, such as a dish name and its price, is more likely to appear together in the same chunk.

2. Increase the chunk overlap

- Add overlap between consecutive chunks so that sentences split across chunk boundaries are included in both chunks. This helps preserve context and improves retrieval accuracy.

Trade-off

The main trade-off is that larger chunks and higher overlap increase storage requirements, retrieval time, and token usage, making the RAG system more computationally expensive.

Conclusion

Increasing chunk size and chunk overlap helps preserve context by keeping related information together, improving retrieval quality. However, this comes at the cost of higher memory usage, slower retrieval, and increased processing costs.

SCENARIO 6 : You are optimising the retrieval component of a RAG-based complaint resolution system for a food delivery platform. The system stores complaint summaries as embeddings in a FAISS index. Query speed is excellent, but retrieved results sometimes include outdated records for resolved complaints that should have been removed. A team member suggests switching to ChromaDB.

Question : Compare FAISS and ChromaDB as vector store options. Explain one scenario where each is the better fit, evaluate whether switching to ChromaDB would solve the outdated-record problem described here, and identify what additional step would be required regardless of which vector store is used .

Ans: FAISS and ChromaDB are both vector stores used to store embeddings and perform similarity search, but they differ in their features.

Comparison of FAISS and ChromaDB

FAISS

- Optimized for fast and efficient similarity search on large embedding collections.
- Best suited for applications where retrieval speed is the highest priority.
- Provides fewer built-in database management features.

ChromaDB

- Stores embeddings along with metadata and offers easier document management (adding, updating, and deleting records).
- Better suited for RAG applications where the knowledge base changes frequently.

When Each is the Better Fit

- FAISS: Best for large, mostly static datasets where high-speed vector search is required, such as searching archived complaint records.
- ChromaDB: Best for dynamic datasets that require frequent updates, additions, or deletions, such as continuously changing complaint records or policy documents.

Would Switching to ChromaDB Solve the Outdated-Record Problem?

Not completely. ChromaDB makes updating and deleting records easier than FAISS, but simply switching vector stores will not automatically remove outdated complaint embeddings. If old records remain in the database, they can still be retrieved.

Additional Step Required

Regardless of whether FAISS or ChromaDB is used, the system must implement a data update and index maintenance process. This includes deleting or updating embeddings for resolved complaints and re-indexing or refreshing the vector store whenever the underlying data changes.

Conclusion:

FAISS is ideal for fast retrieval on static data, while ChromaDB is better for frequently updated knowledge bases. Switching to ChromaDB alone will not solve the outdated-record issue. A proper process for updating, deleting, and re-indexing embeddings is required to keep retrieved results accurate and up to date.